

First: Full adder from 2 half adders

Half adder:

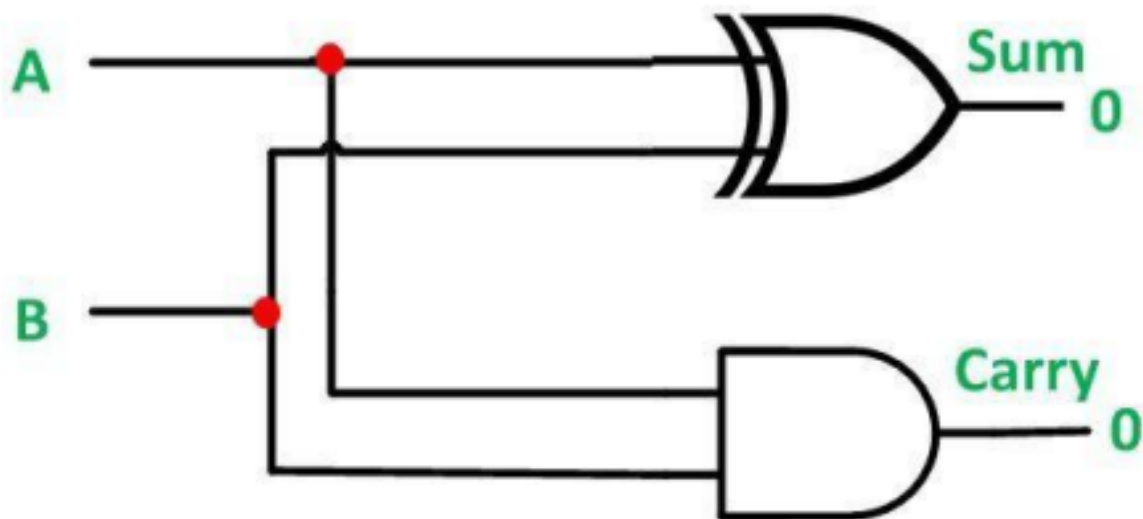
A half adder is a fundamental digital circuit used to add two single-bit binary numbers. It has two inputs (A and B) and two outputs: the sum (S) and the carry (C). The half adder adds the two input bits and produces the sum and carry as outputs.

Operation of a Half Adder:

- Sum (S): The sum output is the XOR (exclusive OR) of the two inputs. The sum is 1 if one of the inputs is 1, but not both.
- Carry (C): The carry output is the AND of the two inputs. It is 1 only if both inputs are 1, which means there is a carry-over to the next bit.

Truth Table for Half Adder:

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Full adder:

A full adder is a more advanced digital circuit than the half adder. It adds three binary numbers: two input bits (A and B) and a carry-in bit (Cin), which comes from the previous stage in a multi-bit addition process. The full adder has two outputs: the sum (S) and the carry-out (Cout).

Operation of a Full Adder:

- Sum (S): The sum output is the XOR of all three inputs (A, B, and Cin). The sum will be 1 if an odd number of the inputs are 1.
- Carry-out (Cout): The carry-out is the OR of the AND results from the combinations of the inputs, meaning it's 1 if two or more inputs are 1.

Truth Table for Full Adder:

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full adder from half adder:

Full adders can be implemented by using 2 half adders and an or gate.

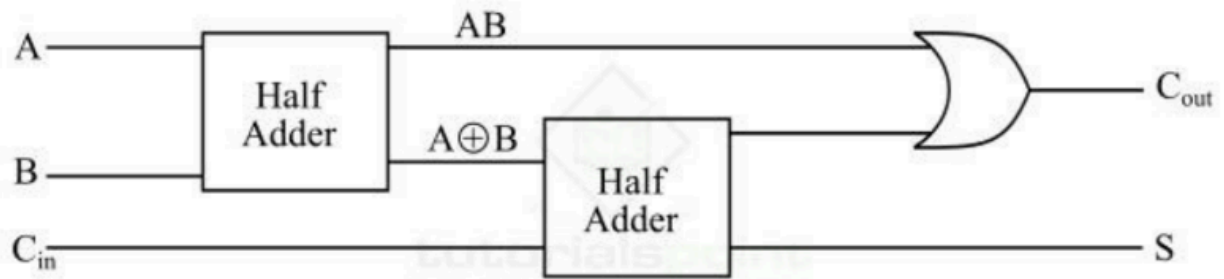
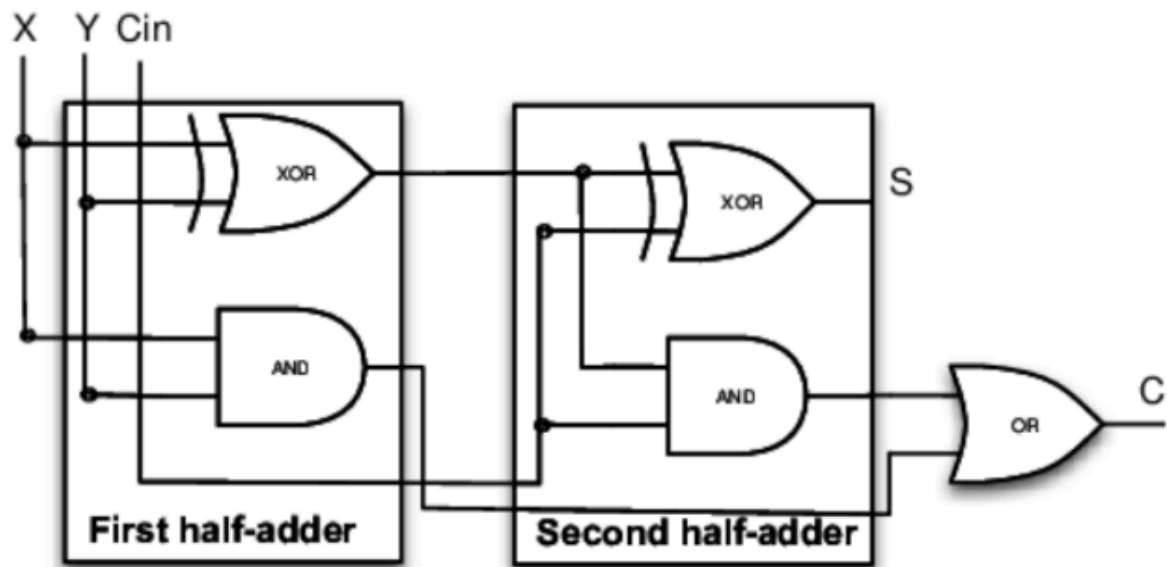


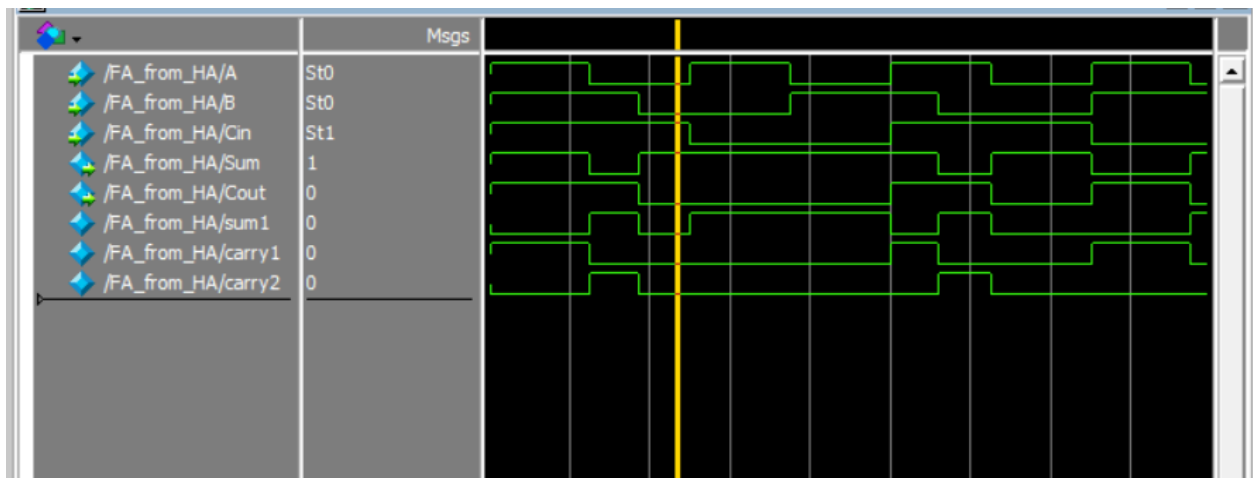
Figure 4 - Block Diagram of Full Adder using Half Adder



Code:

```
1 module HA (  
2     input logic A,  
3     input logic B,  
4     output logic Sum,  
5     output logic Cout  
6 );  
7  
8     assign Sum = A ^ B;  
9     assign Cout = A & B;  
10  
11 endmodule  
12  
13 module FA_from_HA (  
14     input logic A,  
15     input logic B,  
16     input logic Cin,  
17     output logic Sum,  
18     output logic Cout  
19 );  
20     logic sum1, carry1, carry2;  
21     HA ha0 (.A(A), .B(B), .Sum(sum1), .Cout(carry1));  
22     HA ha1 (.A(sum1), .B(Cin), .Sum(Sum), .Cout(carry2));  
23     assign Cout = carry1 | carry2;  
24  
25 endmodule
```

Wave form:



Full adder from two half-adders Verilog code & RTL view:

Quartus Prime Lite Edition - C:/intelFPGA/New folder/Verilog1 - Verilog1

File Edit View Project Assignments Processing Tools Window Help

Verilog1

Project Navigator

Files

- Verilog1.v
- Verilog2.v
- Verilog3.v
- Verilog4.v
- Verilog5.v

Tasks

Compilation

- Task
- Compile Design
- Analysis & Synthesis
- Fitter (Place & Route)
- Assembler (Generate program)
- Timing Analysis
- EDA Netlist Writer

```
1 module Verilog1 (input A, B, Cin,
2   output S, Cout);
3   wire s1, c1, c2;
4   Verilog5 add1 (A, B, s1, c1);
5   Verilog5 add2 (s1, Cin, S, c2);
6   or g1 (Cout, c2, c1);
7 endmodule
```

RTL Viewer - C:/intelFPGA/New folder/Verilog1 - Verilog1

File Edit View Tools Window Help

Netlist Navigator

Verilog1

Verilog5.add1

Verilog5.add2

g1

Cout

S

Cin

0%

00:00:00

Directory

- Section Available
- Functions
- Ice Protocols
- Interface and Controllers
- Sensors and Peripherals
- System Program
- Partner IP